

SciPy and Sparse arrays

(CC-BY: Maria Skeppstedt)

SciPy

Even more operations on arrays than NumPy

Specialized data structures, such as sparse matrices and sparse arrays.

Mathematical operations

Example: Cosine distance

$$1 - \frac{u \cdot v}{\|u\|_2 \|v\|_2}$$

Note 1: Don't use this functionality in the lab, you are meant to implement it using NumPy.

Note 2: Cosine **similarity** and Cosine **distance** is not the same.

Cosine similarity:

$$\frac{u \cdot v}{\|u\|_2 \|v\|_2}$$

Mathematical operations

Example: Cosine distance

$$1 - \frac{u \cdot v}{\|u\|_2 \|v\|_2}$$

```
1 from scipy.spatial import distance
2
3 print(distance.cosine([1, 0], [1, 0])) # cos = 1, 0 degrees
4 print(distance.cosine([1, 0], [0, 1])) # cos = 0, 90 degrees
5 print(distance.cosine([1, 1], [-1, -1])) # cos -1, 180 degrees
```

0.0

1.0

2.0

Sparse arrays

A problem with standard (NumPy-)arrays:
If the data is sparse, they take up a lot of space.

A lot of space is just used for zeros:

For instance, for a large document collection: If columns represent documents, and rows represent words found in the text collection. Then, in a matrix representing word counts in a document, most columns will be zero.
Let's say first document represents "a dog eats a banana"

"a", "and", "all", "ask", "dog", "banana", "eats"

```
1 document_0 = [2, 0, 0, 0, 1, 1, 1, 0, 0 ...]
2
3 document_1 = [0, 4, 0, 0, 0, 0, 0, 0, 0 ...]
4 document_2 = [3, 2, 0, 0, 0, 0, 0, 0, 0 ...]
5 ...
```

Solution: SciPy sparse arrays

Construct sparse array from dense (imagine that `many_zeros` is much larger with many zeros):

```
1 import numpy as np
2 import scipy as sp
3
4 many_zeros = np.array([[0, 0, 9, 0], [0, 0, 0, 8]])
5 print(many_zeros, "\n")
6
7 sparse = sp.sparse.coo_array(many_zeros)
8 print(sparse)
```

```
[[0 0 9 0]
 [0 0 0 8]]
```

```
<C00rdinate sparse array of dtype 'int64'
with 2 stored elements and shape (2, 4)>
```

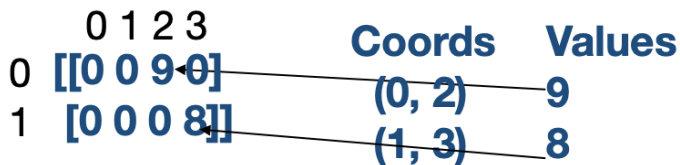
```
Coords Values
```

```
(0, 2) 9
```

```
(1, 3) 8
```

Sparse array

	0	1	2	3		
					Coords	Values
0	[	0	0	9	(0, 2)	9
1	[	0	0	8	(1, 3)	8



Construct efficiently

Better to build, without having to construct the large matrix first:

```
1 #many_zeros = np.array([[0, 0, 9, 0], [0, 0, 0, 8]])
2
3 filled_rows = [0, 1]
4 filled_cols = [2, 3]
5 filled_with = [9, 8]
6 sparse = \
7 sp.sparse.csr_array((filled_with, (filled_rows, filled_cols)))
8 print(sparse, "\n")
9 print(sparse.toarray())
```

<Compressed Sparse Row sparse array of dtype 'int64'
with 2 stored elements and shape (2, 4)>

Coords Values

(0, 2) 9

(1, 3) 8

```
[[0 0 9 0]
 [0 0 0 8]]
```


Adding content

If you include the same coordinate several times, the content will be added

```
1 filled_rows = [0, 1, 0]
2 filled_cols = [2, 3, 2]
3 filled_with = [9, 8, 10]
4 sparse = \
5 sp.sparse.csr_array((filled_with, (filled_rows, filled_cols)))
6 print(sparse, "\n")
7 print(sparse.toarray())
```

<Compressed Sparse Row sparse array of dtype 'int64'
with 2 stored elements and shape (2, 4)>

Coords Values

(0, 2) 19

(1, 3) 8

```
[[ 0  0 19  0]
 [ 0  0  0  8]]
```

Incrementally build

When working with text, you can incrementally build your matrix:

```
1 rows = []
2 columns = []
3 data = []
4
5 rows_for_words = {"a" : 0, "all": 1, "dog": 2,\
6   "eats": 3, "banana": 4}
7
8 documents = [["a", "dog", "eats", "a", "banana"],\
9   ["banana", "eats", "dog"]]
10
11 for document_column, document in enumerate(documents):
12     for word in document:
13         columns.append(document_column)
14         rows.append(rows_for_words[word])
15         data.append(1)
16
17 sparse = \
18 sp.sparse.csr_array((data, (rows, columns)))
19 print(sparse, "\n")
20 print(sparse.toarray())
```

```
1 rows_for_words = {"a" : 0, "all": 1, "dog": 2,\
2   "eats": 3, "banana": 4}
3 documents = [{"a", "dog", "eats", "a", "banana"},\
4   ["banana", "eats", "dog"]]
5 for document_column, document in enumerate(documents):
6     for word in document:
7         columns.append(document_column)
8         rows.append(rows_for_words[word])
9         data.append(1)
```

```
rows [0]
cols [0]
data [1]
****
rows [0, 2]
cols [0, 0]
data [1, 1]
****
rows [0, 2, 3]
cols [0, 0, 0]
data [1, 1, 1]
****
rows [0, 2, 3, 0]
cols [0, 0, 0, 0]
data [1, 1, 1, 1]
****
rows [0, 2, 3, 0, 4]
cols [0, 0, 0, 0, 0]
data [1, 1, 1, 1, 1]
```

```
1 rows_for_words = {"a" : 0, "all": 1, "dog": 2,\n2   "eats": 3, "banana": 4}\n3 documents = [ ["a", "dog", "eats", "a", "banana"],\n4   ["banana", "eats", "dog"]]\n5 for document_column, document in enumerate(documents):\n6     for word in document:\n7         columns.append(document_column)\n8         rows.append(rows_for_words[word])\n9         data.append(1)
```

```
rows [0, 2, 3, 0, 4, 4]\ncolumns [0, 0, 0, 0, 0, 1]\ndata [1, 1, 1, 1, 1, 1]\n*****\nrows [0, 2, 3, 0, 4, 4, 3]\ncolumns [0, 0, 0, 0, 0, 1, 1]\ndata [1, 1, 1, 1, 1, 1, 1]\n*****\nrows [0, 2, 3, 0, 4, 4, 3, 2]\ncolumns [0, 0, 0, 0, 0, 1, 1, 1]\ndata [1, 1, 1, 1, 1, 1, 1, 1]
```

Incrementally build

When working with text, you can incrementally build your matrix:

```
1  ...
2  rows_for_words = {"a" : 0, "all": 1, "dog": 2,\
3    "eats": 3, "banana": 4}
4  documents = [["a", "dog", "eats", "a", "banana"],\
5    ["banana", "eats", "dog"]]
6  for document_column, document in enumerate(documents):
7      for word in document:
8          columns.append(document_column)
9          rows.append(rows_for_words[word])
10         data.append(1)
11 sparse = sp.sparse.csr_array((data, (rows, columns)))
```

Coords	Values
(0, 0)	2
(2, 0)	1
(2, 1)	1
(3, 0)	1
(3, 1)	1
(4, 0)	1
(4, 1)	1

Incrementally build

When working with text, you can incrementally build your matrix:

```
1  ...
2  rows_for_words = {"a" : 0, "all": 1, "dog": 2,\
3    "eats": 3, "banana": 4}
4  documents = [ ["a", "dog", "eats", "a", "banana"],\
5    ["banana", "eats", "dog"] ]
6  for document_column, document in enumerate(documents):
7      for word in document:
8          columns.append(document_column)
9          rows.append(rows_for_words[word])
10         data.append(1)
11 sparse = sp.sparse.csr_array((data, (rows, columns)))
```

```
doc_0 doc_1
[[2  0]   # a
 [0  0]   # all
 [1  1]   # dog
 [1  1]   # eats
 [1  1]]  # banana
```

Caution

Don't do `to_array()` on your very large sparse matrix, then it will take a lot of space.

And don't send your sparse array to another library, which transforms it to a standard array.

```
1 sparse = \  
2 sp.sparse.csr_array((filled_with, (filled_rows, filled_cols)))  
3  
4 sparse.toarray() # Not a good idea for a large sparse matrix
```
